

Power Platform tenant inventory in 60 seconds — a community showcase of the API-first management surface

VerseOps is an open-source, MIT-licensed Windows desktop client that surfaces a complete Power Platform tenant inventory — environments, capacity, solutions, apps, flows, agents, users, and licenses — in a single window. It is built exclusively on official Microsoft surfaces: the **Power Platform Management SDK** (`api.powerplatform.com`), the **BAP capacity API** for the per-environment capacity view that has not yet migrated, **Microsoft Graph**, and the per-environment **Dataverse Web API**.

VerseOps is an independent community project. It is not a Microsoft product, is not endorsed by Microsoft, and carries no Microsoft support contract. It is published as a demonstration of what becomes possible the moment the Power Platform programmability surface is treated as the primary integration point — not as a finished governance suite. No connectors, no service principals, no telemetry, no SaaS backend.

The repository is engineered to clear an enterprise security review on first pass: a CycloneDX SBOM, a CodeQL pipeline, a documented threat model, branch-protection rules, and read-only-by-design API usage are all in the box. Three Authenticode signing paths are documented (self-signed for development, Azure Trusted Signing, and OV/EV) so that adopting organizations can **re-sign the binary with their own enterprise code-signing certificate** before internal distribution.

Source: github.com/SweetsNSavories/VerseOps · MIT.

VerseOps — Power Platform Inventory Console

VerseOps Inventory

ECOSYSTEM GOVERNANCE

Refresh

Cancel

ENVIRONMENTS

716

SYNCED

TENANT DATABASE

383.7 GB

HEALTHY

TENANT FILE

4,472.7 GB

HEALTHY

TOTAL ASSETS

146,462

LIVE

LICENSED USERS

—

GRAPH

Reload from cache

Open trace log

☐ Only my environments

Search environments...

Last refreshed: 2026-05-07 18:05:41

Name	SKU	Status	Region	Version	Ma	Created	Security Group	DB GB	File GB	Log GB	FinOps DB	FinOps File	Instance URL	Env ID
①	Sandbox	Enabled	eastus	9.2.26035.1	OK	2021-01-01	A	1 GB	5.14 / 5 GB	0.000 / 0	0.00 GB	0.00 GB		🔗
②	Sandbox	Enabled	westus	9.2.26035.1	OK	2021-01-01	C	1 GB	5.12 / 5 GB	0.000 / 0	0.00 GB	0.00 GB		🔗
③	Sandbox	Enabled	westus	9.2.26035.1	OK	2026-01-01	C	1 GB	7.27 / 7 GB	0.002 / 0	0.00 GB	0.00 GB		🔗
④	Sandbox	Enabled	eastus	9.2.26035.1	OK	2024-01-01	C	1 GB	9.64 / 10 GB	0.000 / 0	9.87 / 10 GB	0.62 / 1 GB		🔗
⑤	Developer	Enabled	westus	9.2.26035.1	OK	2025-01-01	C	GB	2.33 GB	0.011 GB	0.00 GB	0.00 GB		🔗
⑥	Developer	Enabled	westus	9.2.26033.1	OK	2026-01-01	C	GB	2.38 GB	0.000 GB	0.00 GB	0.00 GB		🔗
⑦	Developer	Enabled	westus	9.2.26035.1	OK	2026-01-01	C	GB	1.92 GB	0.000 GB	0.00 GB	0.00 GB		🔗
⑧	Sandbox	Enabled	westus	9.2.26044.1	OK	2026-01-01	A	GB	8.86 GB	0.000 GB	0.00 GB	0.00 GB		🔗
⑨	Sandbox	Enabled	eastus	9.2.26035.1	OK	2022-01-01	C	1 GB	7.15 / 7 GB	0.000 / 0	0.00 GB	0.00 GB		🔗
⑩	Sandbox	Enabled	eastus	9.2.26035.1	OK	2024-01-01	A	1 GB	6.76 / 7 GB	0.000 / 0	0.00 GB	0.00 GB		🔗
⑪	Sandbox	Enabled	westus	9.2.26035.1	OK	2021-01-01	A	1 GB	7.13 / 7 GB	0.004 / 0	0.00 GB	0.00 GB		🔗
⑫	Developer	Enabled	westus	9.2.26035.1	OK	2025-01-01	C	GB	2.14 GB	0.000 GB	0.00 GB	0.00 GB		🔗
⑬	Developer	Enabled	westus	9.2.26035.1	OK	2025-01-01	C	GB	2.07 GB	0.000 GB	0.00 GB	0.00 GB		🔗
⑭	Developer	Enabled	westus	9.2.26035.1	OK	2026-01-01	C	GB	2.46 GB	0.000 GB	0.00 GB	0.00 GB		🔗
⑮	Developer	Enabled	westus	9.2.26035.1	OK	2026-01-01	C	GB	2.12 GB	0.000 GB	0.00 GB	0.00 GB		🔗
⑯	Sandbox	Enabled	eastus	9.2.26035.1	OK	2025-01-01	C	1 GB	1.82 / 2 GB	0.000 / 0	0.00 GB	0.00 GB		🔗

Loaded 716 envs + 16 tenant capacity rows + 146462 assets from local catalog.

Figure 1 — VerseOps loaded against a live tenant. Per-row capacity (DB / File / Log / FinOps DB / FinOps File GB) is computed from the BAP `$expand=properties.capacity` call; per-env asset counts are joined client-side from the Inventory API result set. Tenant identifiers redacted; everything else is real.

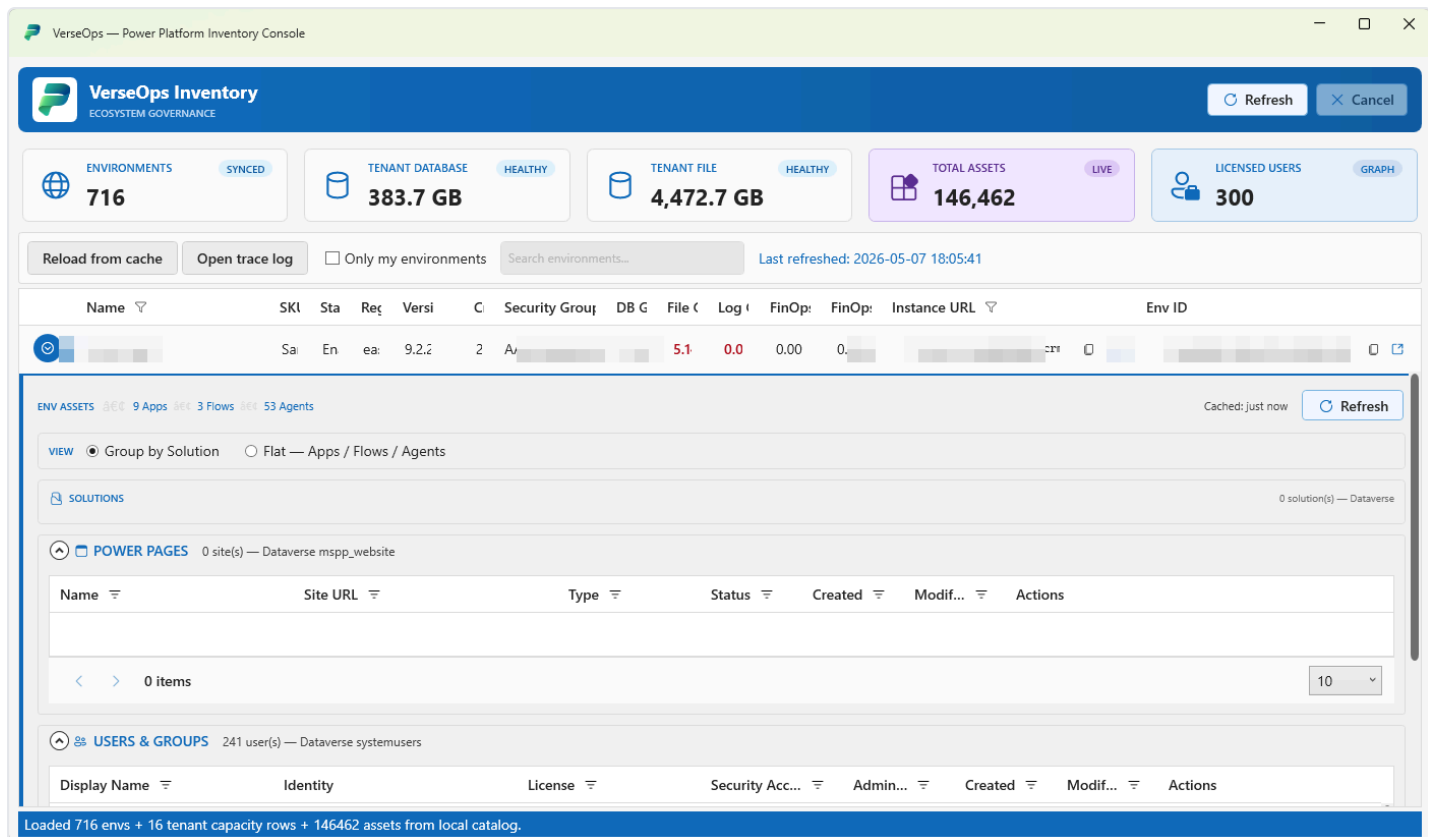


Figure 2 — One environment row expanded. The row-details template fans out the inventory: Solutions / Apps / Flows / Agents (joined from the Inventory API and per-env Dataverse Web API calls), Power Pages sites (`mspg_website` table on the env's Dataverse), and the env's `systemusers` (with their assigned licenses joined from Microsoft Graph). All asset counts (9 / 3 / 53 / 241) are real.

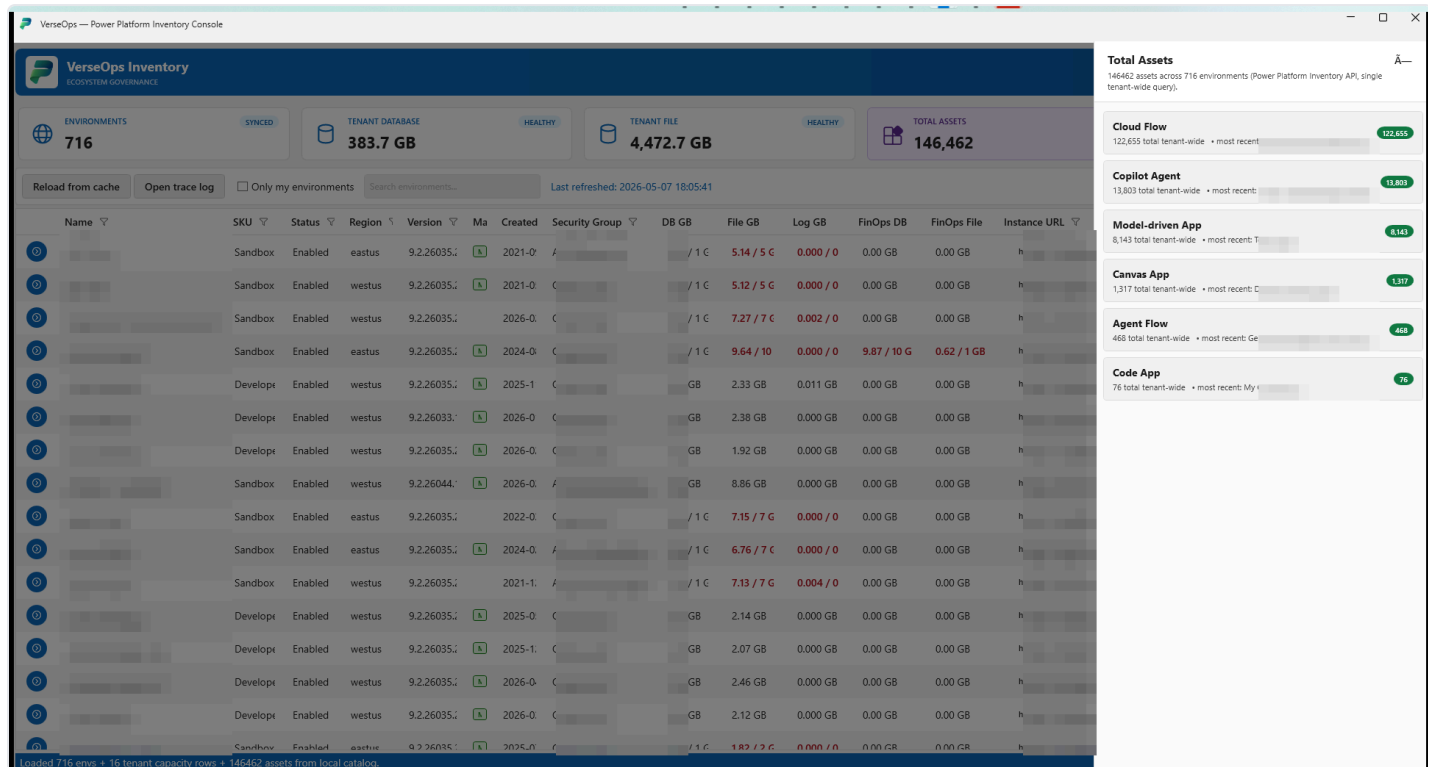


Figure 3 — Total Assets drawer (click the Total Assets KPI tile). The whole panel is fed by a single tenant-wide Inventory API query; the per-type counts are computed client-side from `assetType`. The "most recent" name surfaces the freshest asset of each kind so an admin can sanity-check that the tenant feed is current.

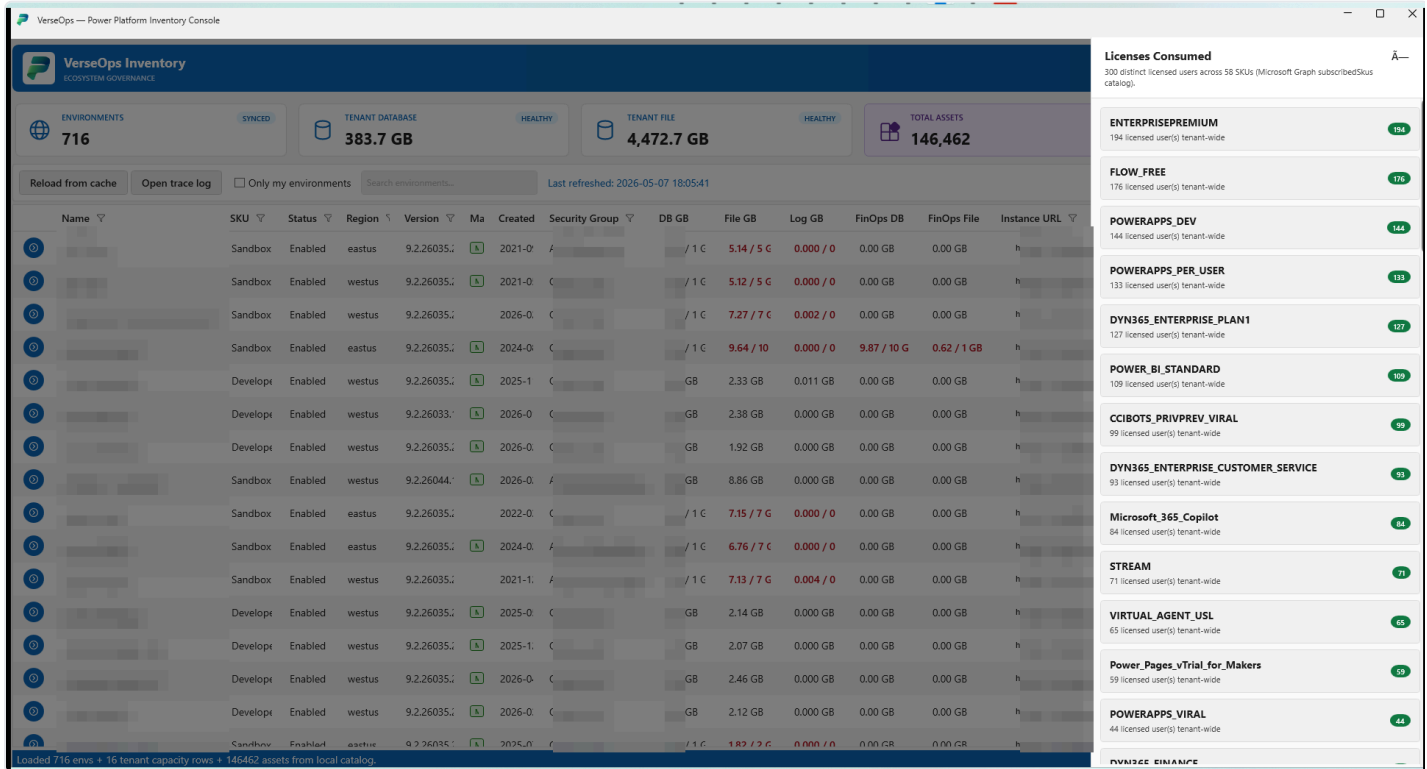


Figure 4 — Licenses Consumed drawer (click the Licensed Users KPI tile). The list is the union of every assigned `servicePlan` from `/users?$select=assignedLicenses` rolled up to the SKU level using the tenant's `subscribedSkus` catalog from Microsoft Graph. SKU codes are public; the only tenant-specific data is the per-SKU seat count on the right.

Why this exists

The recurring questions at the start of every governance cycle are well known:

"How many environments do we actually have? Who owns the apps in them? How much Dataverse capacity is sitting in places no one remembers creating? Which makers left the company three months ago and still own production flows?"

The official answers — Power Platform admin center (PPAC), the **Power Platform inventory** page, and the **Usage** page — already exist and are the right starting point for daily work; they cover the common cases comprehensively. There are still moments, however, when an administrator needs:

1. A **single offline snapshot** they can search, sort, filter, and ship to a stakeholder without exposing the live admin center.
2. A **diff** between this morning and last Friday — *what changed?*

3. **Joined views** that the portal doesn't ship out of the box: per-env capacity × per-env asset count × per-env user count, all in one sortable grid.
4. The **raw JSON** behind every row, one click away, when something doesn't match what the portal shows.
5. A **starting point** — code they can fork, instrument, and turn into the governance tool they actually wanted.

VerseOps targets that long tail. The UI surface is roughly five files; every outbound call is enumerated in [docs/network-endpoints.md](#). The codebase is deliberately small, read-only by design, and positioned as a foundation that adopting teams are expected to fork, instrument, and extend.

How it complements the official "Inventory" and "Usage" pages

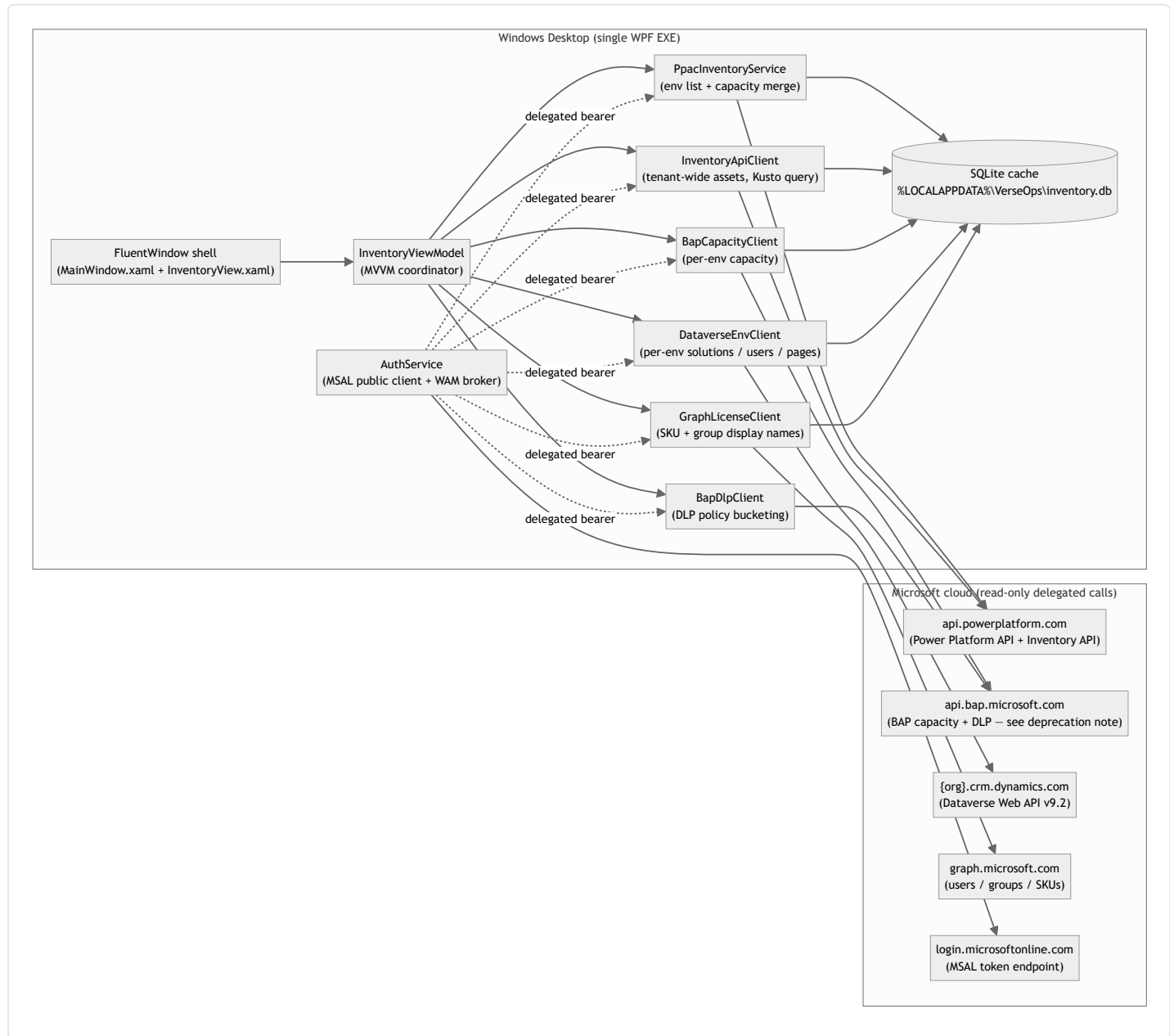
Microsoft's [Power Platform inventory](#) gives administrators a unified view of agents, apps, and flows tenant-wide, refreshed within ~15 minutes. The [Usage page](#) tracks engagement and adoption. Both ship in the admin center today and should be every admin's first stop.

VerseOps is positioned as a **complement, not a replacement**:

Need	PPAC Inventory / Usage	VerseOps
Daily inventory browsing in a portal	✓ Recommended	n/a
Filter / sort / search on any column	✓	✓
Resource-detail drill-in (owner, env, dates)	✓	✓
Export to Excel	✓	✓ (CSV / cache copy)
Capacity (DB / File / Log / FinOps GB) joined per env on the same row as asset count	Partial	✓
One-click "show me the raw Dataverse / PPAC JSON" inspector	✗	✓
Local SQLite cache for offline browsing on a plane / in an air-gapped review	✗	✓
Diff between today's snapshot and yesterday's	✗	✓ (cache-based, on roadmap)
Source you can fork	n/a	✓ MIT, single solution
Telemetry sent to Microsoft / vendor	per Microsoft's data policy	None — zero outbound calls beyond Microsoft's own APIs

If you only ever need 1–4 above, stay in the admin center; it's faster and always up to date. VerseOps shows up when you need 5–11.

Architecture in one diagram



Key architectural properties:

- **Single process, no server-side footprint.** Every call runs in the signed-in user's security context. There is no daemon, no sync job, no message bus. The operating system schedules the network calls; the user triggers a refresh.
- **Two distinct cloud planes.** Management-plane calls (`api.powerplatform.com` , `api.bap.microsoft.com` , `graph.microsoft.com`) are kept separate from data-plane calls (`{org}.crm.dynamics.com` per environment), with audience switching handled centrally by the auth layer.
- **The local SQLite database is the only state.** Removing `%LOCALAPPDATA%\VerseOps\` returns the application to a blank slate. No other persistence exists.

What's actually feasible with the Power Platform API today

Microsoft has been very public about its [shift from a UX-first to an API-first development model for Power Platform programmability](#): new capabilities ship in the API first, then propagate to SDKs, CLI, PowerShell cmdlets, and connectors. The [Programmability and extensibility overview](#) lays out the full toolchain — REST API, .NET SDK ([Microsoft.PowerPlatform.Management](#)), Python SDK, Power Platform CLI, PowerShell cmdlets, and the Power Platform for Admins V2 connector.

VerseOps is a deliberately small showcase of what the **.NET SDK + Inventory API** combination unlocks once you put a UI on it:

Capability	API used	SDK / endpoint
List every environment in the tenant with name / region / SKU / version / security group / default-flag	Power Platform API (PPAC)	<code>Microsoft.PowerPlatform.Management</code> SDK
Per-tenant capacity (DB / File / Log / FinOps DB / FinOps File GB)	Power Platform API (PPAC)	SDK <code>Licensing.Tenant.GetCurrentCapacityAllocations()</code>
Per-environment capacity in one tenant-wide call	BAP capacity (legacy GA)	<code>GET /providers/Microsoft.BusinessAppPlatform/scopes/admin/environments?api-version=2020-10-01&\$expand=properties.capacity</code>
Every canvas app, model-driven app, code app, cloud flow, agent flow, and Copilot Studio agent in the tenant in one POST	Inventory API (preview)	<code>POST https://api.powerplatform.com/resourcequery/resources/query?api-version=2024-10-01</code> (KQL-style query against <code>PowerPlatformResources</code>)
DLP policies + connector classification (Business / Non-Business / Blocked)	BAP Governance v2	<code>GET /providers/PowerPlatform.Governance/v2/policies?api-version=2018-01-01</code>
Per-env solutions / Power Pages sites / system users / roles / app + flow status	Dataverse Web API v9.2	<code>GET {org}/api/data/v9.2/solutions , appmodules , workflows , canvasapps , systemusers , mspp_websites</code>
User license SKU resolution + security-group display names	Microsoft Graph	<code>GET /v1.0/subscribedSkus , /users , /groups , /directoryObjects/getByIds</code>

The headline shape of this: **one tenant-wide POST replaces what used to be N×6 per-environment GETs**. For a tenant with 700 environments, that's the difference between ~4,000 round-trips per refresh and ~10. The same `Microsoft.PowerPlatform.Management` SDK that powers the new admin-center surfaces is the same one your tooling uses — there's no longer a "fast official one and a slow community one".

A note on the BAP API deprecation path

Several BAP routes the community has relied on for years are now in a clear *consolidation* track rather than a *deprecation* one — but the destination is the same. From the official [Versioning and support](#) page:

"The 2020-10-01 Generally available version of Power Platform API is specific to environment management and is also commonly referred to as **Business Application Platform (BAP) API**. The functionality of this set of endpoints are made available in the newer versions of Power Platform API along with many additional features after version 2022-03-01-preview."

In practice, what this means for tools like VerseOps:

BAP route VerseOps uses today	Status (May 2026)	Modern equivalent on <code>api.powerplatform.com</code>
<code>/scopes/admin/environments?expand=properties.capacity</code>	GA (<code>api-version=2020-10-01</code>); functionally superseded but still recommended for tenant-wide capacity	Will move to a Licensing namespace endpoint as parity completes; track Programmability what's new
<code>PowerPlatform.Governance/v2/policies</code> (DLP)	Stable	Watch the new Connectivity / Governance namespace endpoints (e.g. List Connectors , shipped July 2025)
<code>Microsoft.BusinessAppPlatform</code> provider routes	All being mirrored under <code>api.powerplatform.com</code> namespaces (Licensing, EnvironmentManagement, AppManagement, Authorization, Governance, Connectivity)	Use the SDK — Microsoft maintains the mapping for you

Microsoft's official guidance is unambiguous: **use the Power Platform API surface (`api.powerplatform.com`) and one of the official SDKs** (.NET, Python, CLI, PowerShell, [Power Platform for Admins V2 connector](#)) for any new automation. BAP routes won't disappear without a deprecation cycle, but new features ship to `api.powerplatform.com` first and may never come back to BAP.

VerseOps reflects this exactly: every new feature added since April 2026 went to `api.powerplatform.com` , the BAP capacity client is isolated to a single ~150-line file ([BapCapacityClient.cs](#)) so it can be swapped out the moment the per-env capacity surface lands on the new API, and the token-acquisition layer ([AuthService.cs](#)) supports both audiences side by side until that day comes.

Who this helps

The MIT license permits unrestricted internal adaptation; adopting teams are encouraged to fork, re-brand, and re-sign the binary with their own enterprise code-signing certificate as part of internal distribution. Typical adopters include:

- **Power Platform administrators** running quarterly governance reviews who need a single defensible snapshot of current tenant state.
 - **Center-of-Excellence (CoE) leads** who previously relied on the [CoE Starter Kit](#) and are [moving to the in-product Inventory + Usage pages](#), but still require a code-level surface to extend.
 - **FinOps and capacity owners** identifying the ~5% of environments that consume 80% of Dataverse storage, with FinOps DB / FinOps File / Log GB visible on the same row as the environment name.
 - **Mission-critical and regulated workloads** (financial services, healthcare, public sector) where a desktop tool that authenticates as the human administrator, emits zero telemetry, and stores all state locally is materially easier to risk-accept than a SaaS dashboard.
 - **Security and penetration-test teams** who require a reproducible, auditable, signed Windows binary and a clear inventory of what it touches. The [SBOM](#), [SECURITY.md](#), [SIGNING.md](#), and [CodeQL workflow](#) are committed to the repository.
 - **Engineering teams learning the Power Platform API** who want a non-trivial, well-commented .NET sample that exercises every major namespace.
-

Where this could go next

The same API surface that powers VerseOps today can support a substantially richer set of experiences. Candidate directions follow; community input on prioritization is welcome via the [issue tracker](#).

1. An *agentic* governance assistant

Wrap the local SQLite cache + the same auth pipeline behind a Microsoft 365 Copilot agent (or a Foundry agent), and let an admin ask things like:

- *"Which environments grew the most this week and who owns the new flows?"*
- *"List every canvas app with a deprecated connector that's still 'On' in a production env."*
- *"Show me orphaned resources owned by users disabled in Entra in the last 30 days."*

The Power Platform API + Inventory API already returns everything you need to answer these in seconds. The agent surface is just a new face for the same data — and because the cache is local, the agent can run **without ever sending tenant data to a third party**.

2. Periodic snapshots → drift report

A scheduled task that runs `VerseOps.App --refresh --headless` once a day, writes the SQLite snapshot to a versioned folder, and emails a delta. "Today vs yesterday: +12 canvas apps in the Default env, -3 envs decommissioned, capacity climbed 4.1 GB on org-prod-eu."

3. Multi-tenant fan-out for MSPs / consultancies

Same EXE, multiple tenant profiles, side-by-side comparison view. The auth layer already supports `--tenant <guid>`; the cache schema is per-tenant-keyed.

4. Plug-ins for the Inventory API custom queries

The Inventory API's `POST /resourcequery/resources/query` accepts arbitrary KQL-style projections. A plug-in directory of "common admin questions as queries" (orphaned apps, oldest unused flows, premium connector usage by env) could grow organically.

5. Sister tools in Python / TypeScript

The [Python SDK](#) is GA; a Jupyter notebook that mirrors VerseOps' three core panels (env list + capacity + assets) would be ~200 lines and would land instantly with the data-science crowd.

The repository is intended as a working base for these explorations: the foundational ~80% — authentication, caching, paging, retry, redaction, error capture, and theming — is already implemented and exercised against a live ~700-environment tenant. Proposals for any of the directions above can be filed on the [issue tracker](#).

What's in the repository

Everything below is on `main` at github.com/SweetsNSavories/VerseOps, MIT-licensed:

- The single WPF EXE — `VerseOps.App/`
- API clients, one per Microsoft service — `VerseOps.App/Inventory/Services/`
- SQLite catalog schema — `VerseOps.App/Inventory/Sql/schema.sql`
- `README.md` — install, run, build
- `SECURITY.md` — disclosure policy + threat model
- `SIGNING.md` — three publish-with-signature paths (self-signed dev, Azure Trusted Signing, OV/EV)
- `docs/network-endpoints.md` — every outbound host + OAuth scope
- `THIRD-PARTY-NOTICES.md` + `sbom.cdx.json` — full dependency attribution + CycloneDX SBOM
- CI: build, vulnerability scan, CodeQL — `.github/workflows/`
- Branch protection ruleset (PR required, force-push blocked) — `.github/branch-protection.json`

Try it

```
git clone https://github.com/SweetsNSavories/VerseOps.git
cd VerseOps
dotnet build VerseOps.sln -c Release
.\VerseOps.App\bin\Release\net10.0-windows\VerseOps.App.exe
```

Sign in with a tenant admin account (Power Platform Administrator or Dynamics 365 Administrator), click **Refresh**, and the first cold pull populates the local cache. Subsequent launches are instant from the cache; click Refresh again whenever you want a fresh snapshot.

Closing thought

The thesis behind this post is straightforward: an API-first Power Platform management surface puts a complete tenant inventory within reach in roughly 3,000 lines of C#. The official Inventory and Usage pages remain the right tool for daily-driver scenarios. The SDK and Inventory API together cover the long tail — the cases where an organization needs a specific join, requires offline operation, or needs the answer the same week.

VerseOps is offered to the community as that starting point. Issues and pull requests are welcome on the [public tracker](#); security disclosures should follow the process documented in [SECURITY.md](#).

— *Pravin Thatipamula* · maintainer, [VerseOps](#)

References

- [Power Platform inventory](#) — the in-product surface VerseOps complements
- [Power Platform admin center Usage page](#)
- [Programmability and extensibility overview](#) — official tooling map
- [Versioning and support](#) — the BAP-vs-PPAC story
- [Programmability — What's new or changed](#) — monthly release log
- [Power Platform API REST reference \(latest\)](#)
- [Microsoft.PowerPlatform.Management on NuGet](#) — the .NET SDK VerseOps consumes
- [Power Platform for Admins V2 connector](#) — the no-code path to the same API
- [Tutorial: Create a daily capacity report](#) — Microsoft's own end-to-end SDK example
- [Power Platform URLs and IP address ranges](#) — for network allow-lists